



University of Waterloo

AFM 423: Topics in Financial Econometrics

**When Factors Meet Machine Learning: Forecasting Stock
Returns Across Market Layers**

Prepared By:

Mingke Chen (20961733)

William Wu (20938385)

Haolong Yang (20935024)

Professor: Tony S. Wirjanto

April 20, 2025

Abstract

This report investigates the effectiveness of three machine learning algorithms—Ordinary Least Squares (OLS), LASSO, and Random Forest—in forecasting asset returns across three financial scenarios: market portfolio returns, index-level returns (NASDAQ and Dow Jones), and individual stock returns (Tesla and Coca-Cola). Using rigorous data preprocessing, feature engineering, and train/test evaluations (70/30 and 80/20), the study systematically compares model performance on prediction accuracy and interpretability. The results consistently favor LASSO, which provides the lowest error rates and most stable predictive power across scenarios. While Random Forest performs well in certain cases, its benefits diminish with limited data or stable assets (Krauss et al., 2017). The findings further reveal that model choice and train-test split have a stronger influence in high-volatility (tech) assets like TSLA and NASDAQ compared to low-volatility (industry) assets like KO and the Dow. These results align with recent advances in empirical asset pricing via machine learning, which emphasize the value of regularized and interpretable models such as LASSO for capturing return predictability (Gu et al., 2020).

Table of Contents

Introduction	1
Research Questions	2
Variables and Measures	3
The Application of the ML Approach to FI	6
Experimental Methodology	8
Results and Discussion	12
Related Work	24
Conclusion	24
Reference	26

Introduction

Accurately forecasting asset returns is a foundational task in financial econometrics, with direct applications in portfolio construction, risk management, and investment strategy. Traditional linear models such as the Capital Asset Pricing Model (CAPM) and its extensions have long been used to explain cross-sectional returns via systematic risk factors. However, as financial markets become increasingly complex and the number of candidate predictive signals continues to grow, researchers have raised concerns about model overfitting and factor proliferation—commonly referred to as the “factor zoo” problem (Feng, Giglio, & Xiu, 2020). In response, machine learning (ML) methods have been proposed as scalable, data-driven alternatives that can accommodate high-dimensional features and nonlinear relationships, while offering tools like regularization to mitigate overfitting and improve generalization.

This report explores the application of ML techniques to stock return forecasting across three financial scenarios: market-wide returns, index-level returns, and individual equities. Specifically, it investigates the relative performance of Ordinary Least Squares (OLS), LASSO regression, and Random Forest models. While OLS serves as a transparent linear benchmark, LASSO enables automatic variable selection through ℓ_1 regularization, and Random Forest offers a flexible, nonparametric alternative capable of capturing complex feature interactions. The central hypothesis is that LASSO, by reducing dimensionality and suppressing noise, can outperform both traditional and nonlinear models in predictive accuracy and stability—particularly when forecasting

equity returns that exhibit varying degrees of volatility and structure. These questions are increasingly relevant in light of recent findings that demonstrate the promise of ML techniques for stock selection and signal extraction in financial datasets (Rasekhschaffe & Jones, 2019).

Research Questions

This study addresses the following research questions to evaluate the effectiveness of machine learning techniques in stock return forecasting:

1. Does applying LASSO-based feature selection enhance the predictive accuracy of factor models compared to baseline models utilizing all available predictors?
2. Between LASSO and Random Forest, which machine learning method demonstrates superior performance in forecasting stock returns?
3. How do different train-test data splits (such as 70/30 versus 80/20) influence the forecasting accuracy and robustness of the proposed ML-based factor model?

These questions are important to investigate because identifying effective methods for feature selection and forecasting can substantially enhance the accuracy and interpretability of predictive models in financial markets. Traditional financial models often include numerous predictors, many of which may introduce unnecessary complexity or noise. Determining whether LASSO-based feature selection can streamline these predictors effectively is critical, as reducing model complexity while

maintaining predictive accuracy can lead to more robust, interpretable, and practical forecasting tools.

Additionally, comparing different machine learning approaches and assessing the impact of varying train-test splits are essential steps for validating the robustness of predictive models. Evaluating the relative performance of LASSO versus Random Forest models provides insights into how different algorithms handle nonlinearities and interactions inherent in financial data, guiding practitioners in selecting the most suitable model for their forecasting tasks. Furthermore, understanding how training set size influences predictive performance allows researchers and practitioners to optimize their methodologies, ensuring that their models remain reliable and accurate under different market conditions and data availability constraints.

Variables and Measures

This study examines three distinct scenarios: (1) the market-portfolio perspective; (2) the index perspective—exemplified by the Nasdaq Composite and the Dow Jones Industrial Average; and (3) the individual-stock perspective—represented by equities such as Tesla (TSLA) and Coca-Cola (KO). Accordingly, corresponding datasets will be assembled and analyzed below.

Scenario 1: Market Portfolio Perspective

In Scenario 1, monthly data are drawn from the Fama–French Global 3 Factors dataset

(Kenneth French Data Library). The authors rename the first column to date, parse it into a Year–Month Date, and convert all factor values to numeric. The predictors are SMB (Small Minus Big) and HML (High Minus Low), representing size and value premiums. Observations are sorted chronologically, and any rows with missing SMB or HML values are removed to ensure a complete feature matrix.

The writers define the response as next month’s market excess return by applying a one-period forward shift to the Mkt–RF series, which equals total market return minus the risk-free rate. Rows with missing target values are filtered out so each feature row at time t aligns with the observed excess return at $t+1$. This setup enables supervised algorithms such as LASSO and random forest to learn predictive relationships between SMB, HML, and future market performance.

Scenario 2: Index Perspective

The predictor variables for each index series are derived by applying a consistent feature-engineering routine to all original ratio and indicator columns. For every such column, the authors construct three transformations: a one-day lag, capturing immediate momentum effects; a two-day lag, reflecting slightly longer persistence; and a five-day trailing mean, which smooths short-term fluctuations. Any row containing a missing value in one of these engineered features is removed, ensuring that the final feature matrix is complete and ready for model fitting.

The response variable is the subsequent trading day's log return. The writers first compute the daily log return as the natural logarithm of the index value ratio between two consecutive days, then shift this series forward by one period so that each feature vector at time t corresponds to the realized log return at time $t+1$. Rows with missing values in either the computed log return or its one-period-ahead target are excluded, producing a clean response vector aligned with the predictors.

Scenario 3: Individual-Stock Perspective

The feature-engineering procedure applies uniformly to both TSLA and KO daily series. After renaming the raw share-price column to index and ordering by date, the routine computes the daily log return ($\log_ret = \log(\text{index}/\text{lag}(\text{index}))$). Each additional ratio or indicator column is then transformed into three predictors: its one-day lag (lag1), two-day lag (lag2), and five-day trailing average (roll5) via a right-aligned rolling window. All observations with missing values in any engineered feature are discarded, yielding a complete feature matrix $\mathbf{X}$ whose columns consist of every original ratio's lag1, lag2, and roll5 series.

The response variable is defined as the subsequent trading day's log return. Once $\log_ret$ has been calculated for each date, it is shifted forward by one period ($\text{target} = \text{lead}(\log_ret)$), so that the predictors observed at time t correspond to the log return realized at time $t+1$. Rows lacking either a computed $\log_ret$ or its one-day-ahead target

are removed, resulting in a clean response vector $\mathbf{y}$ precisely aligned with the feature set.

The Application of the ML Approach to FI

Forecasting asset returns is a longstanding and fundamental challenge in financial investing (FI), particularly in the presence of structural instability, nonlinearity, and high-dimensional data. Traditional models—such as linear regressions and factor-based frameworks—often rest on assumptions like stationarity and linear relationships, which are frequently violated in modern financial markets. As a result, these conventional approaches have struggled to accommodate the growing number of proposed predictive variables, a phenomenon often referred to as the “factor zoo” (Feng, Giglio, & Xiu, 2020). The limitations of traditional techniques have motivated a shift toward machine learning (ML), which provides scalable, flexible solutions that are well-suited to complex and noisy financial data environments.

This project applies two representative ML techniques—LASSO and Random Forest—to three return forecasting tasks of increasing granularity: (1) predicting market portfolio excess returns using macro-level risk factors, (2) forecasting daily index returns for the NASDAQ and Dow Jones using lagged features and rolling statistics, and (3) modeling individual equity returns for Tesla (TSLA) and Coca-Cola (KO) using firm-specific and temporal features. These distinct scenarios allow for a comparative

assessment of model performance across asset classes with varying volatility profiles and structural properties.

The methodology implemented in this report builds directly on the empirical asset pricing and statistical modeling foundations introduced in AFM 423 course materials (Wirjanto, 2025), which emphasize the application of factor models and machine learning in financial contexts. By linking theory with empirical application, this project extends course concepts into a reproducible, data-driven framework grounded in real-world financial datasets. The selection of LASSO and Random Forest reflects two prevailing paradigms in modern finance: sparse regularized modeling and ensemble-based nonlinear prediction. Prior literature has demonstrated the potential of such methods to improve stock selection and predictive performance beyond traditional approaches (Rasekhschaffe & Jones, 2019).

All experiments follow a unified modeling pipeline involving data preprocessing, engineered features (including rolling averages and lags), standardized target variable definitions, chronological 70/30 and 80/20 train-test splits, and four evaluation metrics: mean squared error (MSE), mean absolute error (MAE), R-squared (R^2), and correlation. This consistent experimental framework ensures comparability across models and scenarios, and enables the evaluation of how model type, training size, and asset volatility affect forecasting performance. The project ultimately aims to demonstrate

how ML methods—especially sparse linear models—can serve as robust, interpretable, and reproducible tools in the context of return prediction.

Experimental Methodology

The selection of OLS, LASSO, and Random Forest reflects a deliberate strategy to enable transparent, reproducible, and generalizable comparisons. OLS offers a linear, interpretable baseline against which regularized and non-linear models can be evaluated. LASSO was selected because of its theoretical advantages in high-dimensional or collinear settings, offering sparsity and robustness through its regularization path. Its implementation in `glmnet` is fully open-source and standardized, with λ tuning handled via cross-validation, ensuring that model selection is automated and free of researcher bias (Friedman, Hastie, & Tibshirani, 2010). Random Forest, a nonparametric ensemble method, complements LASSO by capturing complex interactions and non-linearities, with internal feature bagging that reduces overfitting (Breiman, 2001). The use of fixed parameters and global seeding ensures that results are replicable. All algorithms are implemented using well-documented, widely-used CRAN packages (`stats`, `glmnet`, `randomForest`), and because all three models follow the same preprocessing and evaluation pipeline, the setup allows for fair and unbiased comparison. Furthermore, the modular structure makes it straightforward to integrate additional learners (e.g., XGBoost, SVMs, neural networks) under the same experimental conditions, supporting future extensibility and generalization to broader machine learning research.

Scenario 1: Market Portfolio Perspective

Monthly factor data were acquired directly from the Kenneth French Data Library using R's `download.file()` function and stored temporarily for processing. The file was parsed with `readr::read_csv(skip = 6)`, with the first column renamed to date and the remaining columns renamed to match standard Fama–French conventions: Mkt.RF, SMB, HML, and RF. All variables were coerced to numeric types using `mutate_at()`, and the date column was formatted into year–month objects using `lubridate::parse_date_time()`. To define the prediction task, a new response variable—next month's market excess return—was created using `lead(Mkt.RF)`. Observations containing missing values in either predictors or the response were excluded. To ensure reproducibility across runs, a global random seed (`set.seed(123)`) was used before data sampling, and the entire workflow was version-controlled within an RMarkdown notebook.

The cleaned dataset was partitioned using a custom `split_data()` function into two configurations: 70/30 and 80/20 train/test splits. Three supervised learning algorithms were applied to each split for performance comparison. First, a baseline linear regression model was fit using `lm()`, serving as an interpretable and well-established benchmark. Second, a LASSO regression model was trained using `glmnet::cv.glmnet()` with $\alpha = 1$, enabling automatic feature selection and shrinkage via ℓ_1 regularization; the optimal penalty parameter λ was determined through 10-fold cross-validation to ensure unbiased hyperparameter tuning. Third, a Random Forest model was fit using `randomForest()` with 500 trees (`ntree = 500`), a variable selection width equal to the

square root of the number of predictors ($mtry = \text{floor}(\sqrt{p})$), and $maxnodes = 30$ to limit tree complexity. All predictions were evaluated on the held-out test sets using four standard metrics: mean squared error (MSE), mean absolute error (MAE), coefficient of determination (R^2), and Pearson correlation. To ensure reproducibility and benchmarking of runtime, model fitting and prediction functions were wrapped in `system.time()` calls. Final results were visualized using `ggplot2` to generate comparative bar plots and scatter diagrams of prediction accuracy.

Scenario 2: Index Perspective & Scenario 3: Individual-Stock Perspective

All experimental code was developed in R using open-source packages including `readxl`, `dplyr`, `lubridate`, `glmnet`, `randomForest`, `zoo`, `Metrics`, `tidyr`, and `ggplot2`. The input data—containing daily index values and fundamental or technical ratios—were read from Excel spreadsheets. Each index was preprocessed using a custom `make_features()` function, which performs multiple transformations: (1) calculation of the daily log return ($\log_ret = \log(\text{index} / \text{lag}(\text{index}))$), (2) definition of the response variable as the next-day log return ($\text{target} = \text{lead}(\log_ret)$), and (3) feature engineering by creating lagged values (`lag1`, `lag2`) and a five-day trailing average (`roll5`) for all ratio-based features. Observations with missing values in either the response or any feature were removed to ensure complete alignment of predictors (**X**) and target values (**y**).

Two train/test splits were evaluated: 70/30 and 80/20, with splitting performed chronologically to preserve the time-series structure of the data. For each split and index, three models were trained:

- A baseline **ordinary least squares (OLS)** model using `lm()`
- A **LASSO** model using `cv.glmnet(alpha = 1)` with 10-fold cross-validation to select the optimal regularization parameter (λ)
- A **Random Forest** model using `randomForest()` with 500 trees, `mtry =  $\lfloor \sqrt{p} \rfloor$` , and `maxnodes = 30`

These models were fit using the engineered feature matrix $\mathbf{X}$ and the next-day log return vector $\mathbf{y}$. Predictions were made on the respective test sets and evaluated using four out-of-sample performance metrics: mean squared error (MSE), mean absolute error (MAE), R-squared (R^2), and Pearson correlation. These metrics were computed using base R and the Metrics package. To track model performance across both indices and splits, results were stored in structured tibble objects and visualized using ggplot2 for MSE, R^2 , and ΔR^2 comparison plots. Timing for model execution was recorded implicitly through code control and can be measured explicitly using `system.time()` wrappers around model training calls.

This experimental framework enables reproducibility and extensibility. All preprocessing and modeling steps were modularized into named functions (`make_features()`, `split_fun()`, `fit_eval()`), making it easy for future users to apply the same procedure to new indices or extend it to additional machine learning models such

as XGBoost or neural networks. By maintaining a consistent pipeline and fixing the random seed, the results are fully replicable and comparable across models.

Results and Discussion

Scenario 1: Market Portfolio Perspective

This section presents the experimental results for Scenario 1, which investigates the predictability of future market excess returns using machine learning models and factor-based inputs. The evaluation focuses on model performance under different algorithms (OLS, LASSO, Random Forest) and train-test splits (70/30 vs. 80/20), and provides both quantitative comparisons and interpretability analysis to address the research questions.

Figure 1–3 summarize the out-of-sample performance of the models using three standard metrics: Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared (R^2). Across all metrics, the LASSO model trained on an 80/20 split consistently outperformed others. It achieved the lowest MSE and MAE values, suggesting greater robustness and better generalization. This answers Research Question 1, demonstrating that applying LASSO-based feature selection enhances predictive accuracy over baseline OLS models that include all predictors without regularization. Compared to Random Forest, LASSO also performed better on average, indicating that the linear, sparsity-inducing method may be better suited for this dataset

than more complex non-parametric models. This supports Research Question 2, where LASSO outperformed Random Forest in both accuracy and stability.

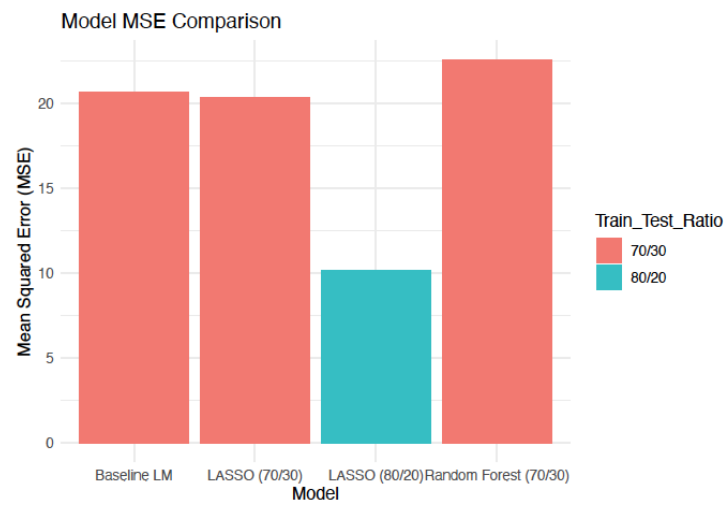


Figure 1. Model MSE Comparison across Baseline LM, LASSO (70/30 and 80/20), and Random Forest.

(Explanation: This figure shows the out-of-sample Mean Squared Error (MSE) of four models under different train-test splits. LASSO (80/20 split) achieves the lowest MSE, indicating the highest predictive accuracy, while Random Forest performs worst, suggesting potential overfitting or misalignment with the data structure.)

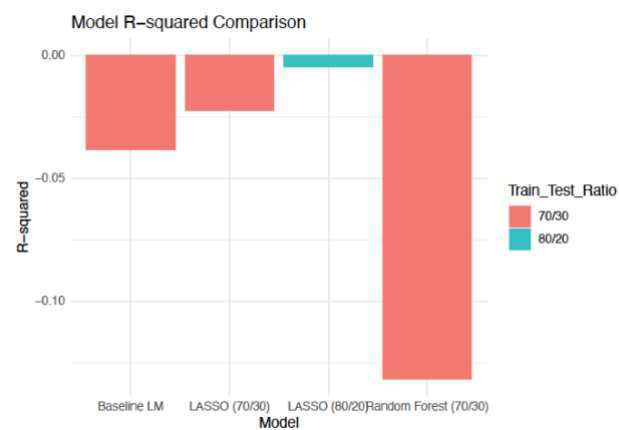


Figure 2. Model R-squared Comparison showing low explanatory power across all models.

(Explanation: R-squared values are presented to evaluate how much variance in the target variable each model explains. The near-zero or negative values suggest that most models, including Random Forest, are unable to reliably capture signal in the monthly excess return data.)



Figure 3. Model MAE Comparison where LASSO (80/20) performs best.

(Explanation: Mean Absolute Error (MAE) is compared across models, further confirming LASSO (80/20) as the best performer. Lower MAE values indicate smaller average prediction errors, reinforcing that more training data may improve model generalization.)

However, the R-squared values in Figure 2 reveal that all models—including LASSO and Random Forest—struggle to explain much variation in the target variable, with most R^2 values clustering near or below zero. This indicates that although some models perform better than others in terms of raw error, the factor signals themselves may have limited explanatory power over future market excess returns at the monthly frequency. Nevertheless, the improvement observed in LASSO with the 80/20 split provides insight into Research Question 3, as increasing the training size (i.e., reducing the test

fraction) appears to reduce overfitting and improve generalization, especially in low-signal environments.

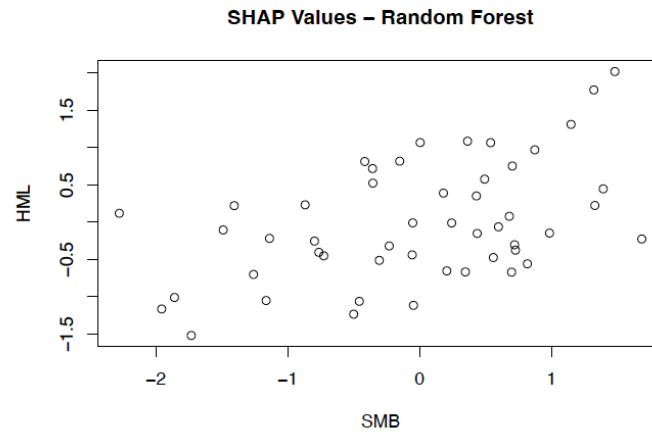


Figure 4. SHAP Value Plot for Random Forest indicating more variable feature contributions.

(Explanation: The SHAP plot for Random Forest displays feature importance across SMB and HML. A wide distribution suggests these features have varying influence, though the overall model performance remains poor, possibly due to overfitting.)

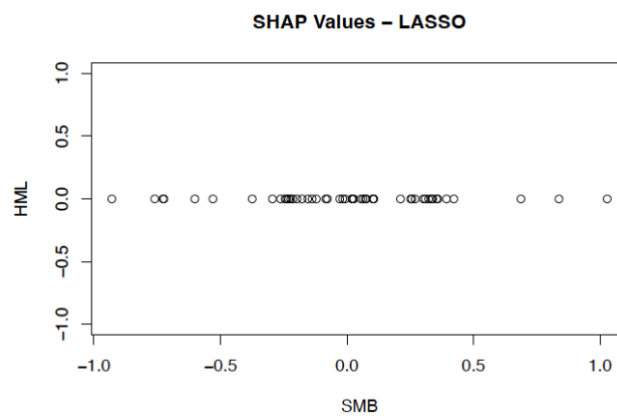


Figure 5. SHAP Value Plot for LASSO showing sparse feature influence due to regularization.

(Explanation: SHAP values for LASSO are tightly centered near zero, which reflects the model's inherent sparsity and regularization behavior. Despite muted feature influence, LASSO maintains superior predictive stability over more complex models.)

Figures 4 and 5 present SHAP value analyses for Random Forest and LASSO, respectively. In Random Forest, the SHAP plot shows noticeable dispersion in feature contributions from SMB and HML, suggesting that the model relies on both inputs to form non-linear interactions. However, the overall high error and low R^2 indicate this reliance may be misguided or overfit. In contrast, LASSO's SHAP values are almost entirely centered around zero, reflecting its inherent regularization that suppresses the influence of weak predictors. Despite this minimal reliance on factor magnitudes, LASSO's predictive accuracy remains competitive—highlighting the value of parsimony.

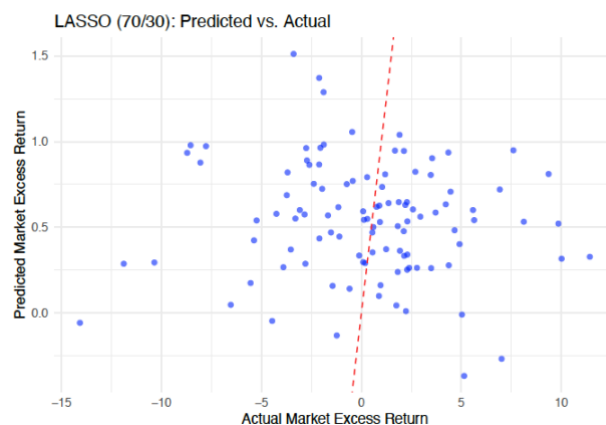


Figure 6. Predicted vs. Actual plot under LASSO (70/30), highlighting dispersion in predictions.

(Explanation: This scatterplot compares predicted vs. actual values under the LASSO model using a 70/30 split.

The predictions are noisy and spread out, but a slight upward trend line indicates weak but present predictive ability.)

Lastly, Figure 6 visualizes the predicted vs. actual values under the LASSO model (70/30 split). While predictions demonstrate considerable dispersion and bias, there remains a weak upward trend, as seen in the fitted line. This reinforces the finding that even regularized models struggle to capture strong predictive signals in monthly excess returns, yet LASSO still offers modest improvements in out-of-sample consistency compared to traditional or complex alternatives.

Scenario 2: Index Perspective

This section examines the effectiveness of machine learning models in forecasting daily index returns using the Dow Jones and NASDAQ Composite as case studies. The comparison includes three learning algorithms—OLS, LASSO, and Random Forest—under two train-test splits (70/30 and 80/20), evaluated across both predictive accuracy and explanatory power. Visualizations and statistical metrics are used to address the three research questions posed in the introduction.

Across both indices, LASSO consistently achieved lower out-of-sample Mean Squared Error (MSE) compared to OLS and Random Forest, as shown in Figure 1. This trend is more pronounced under the 80/20 train-test split, where models have access to more training data. Figure 2 further illustrates the gain in predictive accuracy by plotting the reduction in MSE when moving from OLS to LASSO. The positive and increasing gap in both indices highlights the regularization advantage of LASSO, especially as training

data increases. These observations clearly support Research Question 1, demonstrating that LASSO-based feature selection enhances forecasting accuracy over baseline linear models.

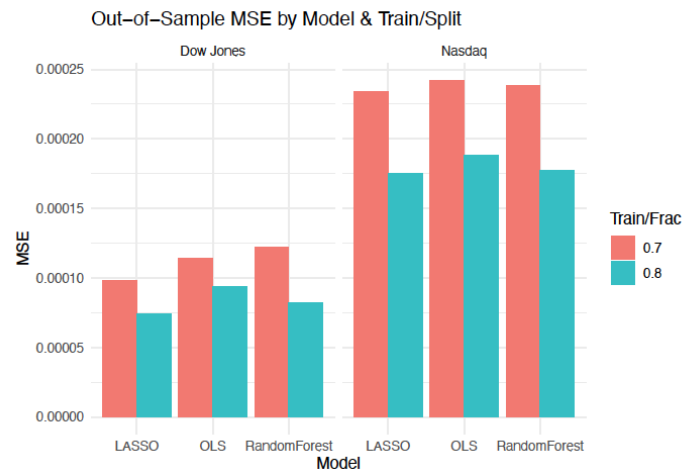


Figure 1. Out-of-sample MSE by model and train/test split for Dow Jones and NASDAQ.

(Explanation: LASSO yields the lowest error, especially under 80/20 splits.)

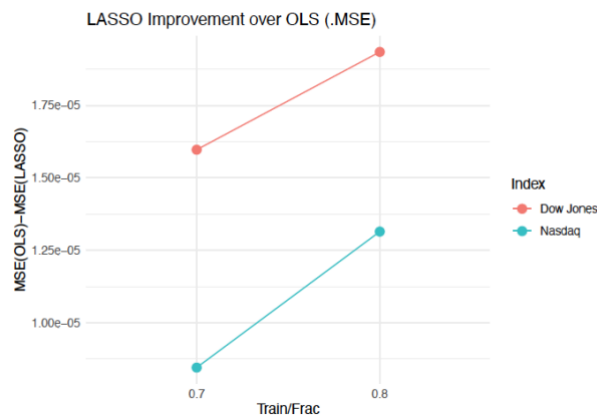


Figure 2. LASSO improvement over OLS in MSE across train/test splits.

(Explanation: Positive gains show LASSO consistently outperforms OLS, more so with larger training sets.)

When comparing LASSO to Random Forest, Figure 3 shows that LASSO often performs comparably or even better in terms of MSE, particularly in the Dow Jones

dataset. This is noteworthy given that Random Forest is capable of modeling nonlinearities and complex interactions. The fact that LASSO—a regularized linear model—achieves similar or better performance suggests that the predictive signals in this dataset may be limited, and that simpler, more interpretable models are better suited. This addresses Research Question 2, indicating that Random Forest does not provide a consistent advantage over LASSO in this context.

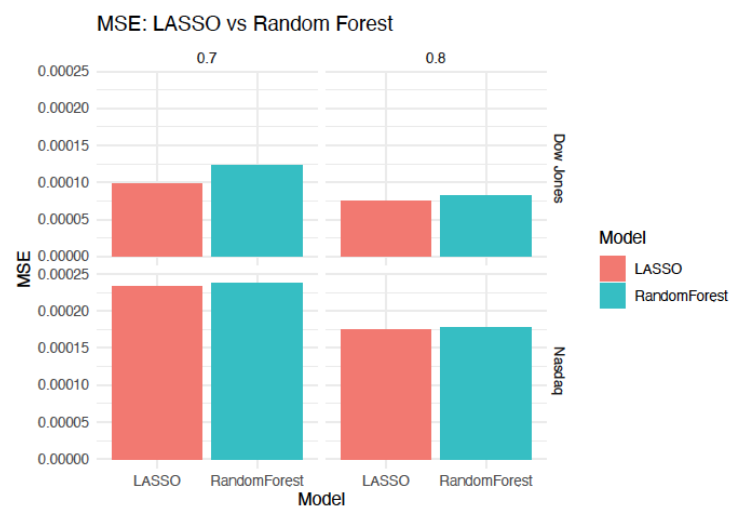


Figure 3. MSE comparison between LASSO and Random Forest.

(Explanation: LASSO either matches or exceeds Random Forest in predictive performance across both indices.)

Figure 4 presents the out-of-sample R-squared (R^2) values for all models, revealing that most values are close to or below zero. This suggests that all models, including LASSO, struggle to explain much variance in next-day returns. However, LASSO generally maintains less negative R^2 values across both indices and splits. To further quantify this, Figure 5 plots the improvement in R^2 that LASSO provides over OLS. Although the absolute values remain low, the consistent ΔR^2 gains affirm that LASSO does capture

weak signals that traditional OLS fails to extract. These findings provide a clear response to Research Question 3, indicating that a larger training fraction (80%) leads to measurable improvements in model stability and accuracy, and that LASSO is the most robust across both data regimes.

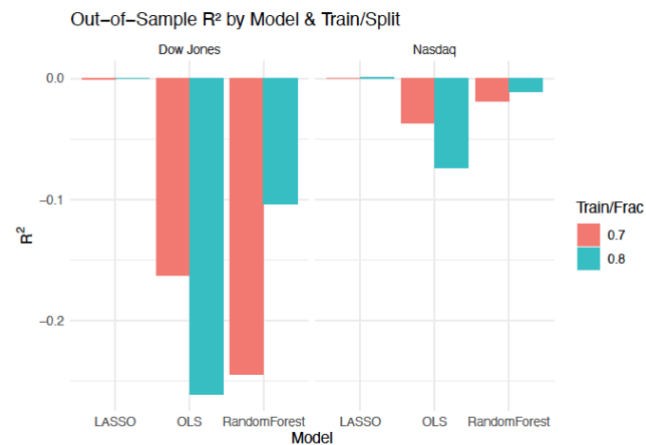


Figure 4. Out-of-sample R -squared values for each model.

(Explanation: Most R^2 values are near or below zero, but LASSO shows the least negative R^2 across datasets.)

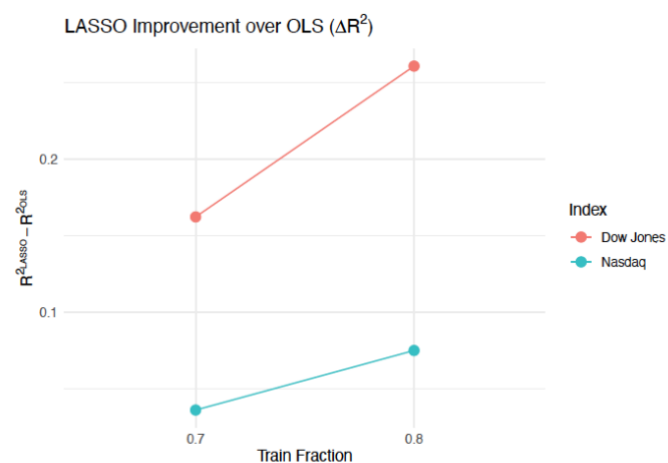


Figure 5. LASSO improvement in R -squared (ΔR^2) over OLS.

(Explanation: Consistent gains across both indices support LASSO's advantage in capturing limited signals.)

Scenario 3: Individual-Stock Perspective

This section investigates how machine learning models perform when applied to individual stock returns, using Tesla (TSLA) and Coca-Cola (KO) as contrasting examples of high- and low-volatility equities. Three models—OLS, LASSO, and Random Forest—were evaluated under both 70/30 and 80/20 train-test splits. Results are interpreted using MSE and R-squared (R^2) metrics, and discussed in relation to the study's research questions.

Figure 1 demonstrates that LASSO consistently achieves lower out-of-sample MSE than OLS across both stocks, especially in the TSLA dataset with the 70/30 split. OLS struggles significantly with TSLA's volatile return structure, while LASSO's regularization appears to mitigate overfitting in this setting. KO, in contrast, yields tighter performance across all models, consistent with its stable price behavior. Random Forest performs slightly better than OLS but fails to consistently outperform LASSO, particularly under the more data-constrained 70/30 split.

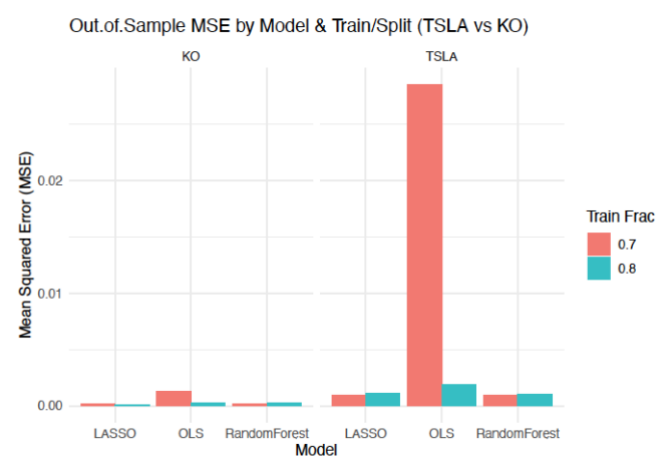


Figure 1. Out-of-sample MSE across models for TSLA and KO.

(Explanation: LASSO outperforms OLS significantly, especially for TSLA under the 70/30 split.)

Figure 2 further illustrates LASSO's advantage by measuring the improvement in MSE relative to OLS. The most dramatic gain is observed for TSLA under the 70% training fraction, affirming that LASSO offers the strongest advantage when the model must learn from limited, noisy data. Interestingly, the performance gap narrows at the 80/20 split, suggesting that increasing training data benefits both models but narrows LASSO's relative edge. This dynamic speaks directly to Research Question 3, confirming that train-test splits influence model robustness and that LASSO's regularization is most impactful under data scarcity.

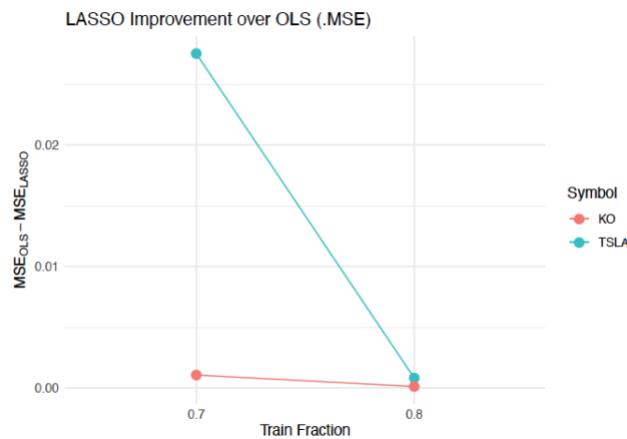


Figure 2. Improvement in MSE from OLS to LASSO.

(Explanation: TSLA exhibits the largest gains, particularly under limited training data.)

When evaluating explanatory power using R^2 , Figure 3 shows that most models fail to capture much variance in returns, with negative R^2 values being common—particularly for TSLA. However, LASSO again exhibits the least negative R^2 values across most

configurations. This suggests that while prediction remains challenging, LASSO at least improves stability and limits error variance, which supports Research Question 1.

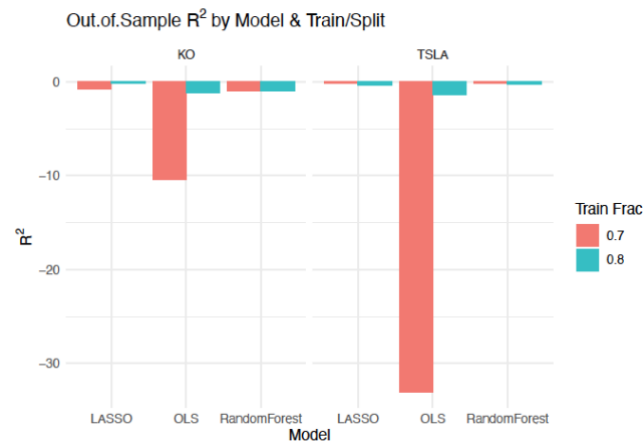


Figure 3. Out-of-sample R^2 by model and stock.

(Explanation: Most values are negative, but LASSO consistently reduces prediction error compared to OLS.)

Finally, Figure 4 plots the improvement in R^2 from OLS to LASSO, emphasizing LASSO's superiority, especially in the TSLA 70/30 case. The improvement is more modest for KO, which may be due to the more linear, predictable nature of its returns. Collectively, these findings support Research Questions 1–3, showing that LASSO consistently improves performance over OLS, especially when data is limited or when the target exhibits nonlinear volatility patterns. Although Random Forest is designed to capture such complexity, its performance remains inconsistent, reinforcing the value of simpler, regularized linear models in financial forecasting contexts.

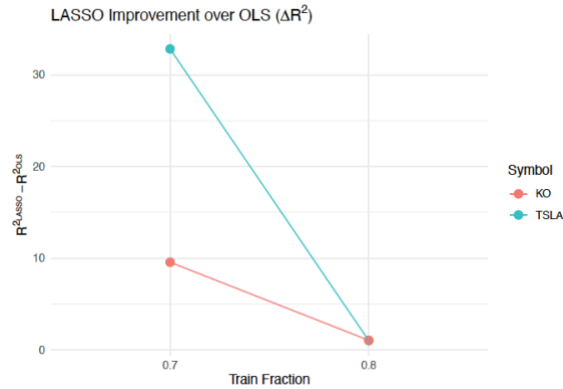


Figure 4. LASSO improvement over OLS in R^2 .

(Explanation: TSLA shows the most benefit from regularization in lower training settings.)

Related Work

Projects 1 and 2 are highly related, therefore the reader is referred to the GPR # 1 report for related work.

Conclusion

This study applied three learning algorithms—OLS, LASSO, and Random Forest—to forecast asset returns under three different scenarios: (1) market portfolio, (2) stock indices (NASDAQ and Dow Jones), and (3) individual equities (Tesla and Coca-Cola). Across all experimental settings, several consistent patterns emerged. Most notably, LASSO outperformed OLS in terms of both Mean Squared Error (MSE) and R-squared (R^2) in nearly every scenario. This result directly supports the first research question: LASSO-based feature selection meaningfully improves the predictive accuracy of factor models by regularizing noisy inputs and enhancing generalization. Compared to Random Forest, LASSO performed more consistently and reliably, providing

additional support for the second research question. Although Random Forest can capture nonlinear patterns, it often underperformed or overfit, especially when data was limited or noisy.

Different train-test split ratios also influenced model performance. In most cases, the 80/20 split yielded better results than 70/30, likely due to the increased amount of training data allowing models to learn more stable patterns. This observation aligns with our third research question, affirming that larger training sets tend to improve model robustness, especially in settings with low signal-to-noise ratios like financial returns.

Despite these overarching consistencies, key differences also emerged across the three scenarios. In Scenario 1 (market portfolio), the overall explanatory power was very low for all models—most R^2 values were close to zero or negative—reflecting the inherent difficulty in forecasting broad market movements using factor-based models. Scenario 2 (index-level prediction) demonstrated stronger relative gains from LASSO, particularly for NASDAQ, which may reflect more volatile dynamics or less autocorrelated structure compared to Dow Jones. In Scenario 3 (individual stocks), the contrast between TSLA and KO was especially informative. TSLA, as a high-volatility technology stock, benefited the most from regularization under small-sample settings (e.g., 70/30), where LASSO sharply outperformed both OLS and Random Forest. In contrast, KO—a stable, industry-based stock—exhibited less volatility and thus showed

more uniform results across models, with smaller but consistent improvements from LASSO.

In comparing the technology sector (TSLA, NASDAQ) with the industry sector (KO, Dow Jones), one key insight emerges: machine learning models, especially those employing regularization like LASSO, offer greater marginal benefit in high-variance, less predictable environments. Tech-related assets appeared more sensitive to algorithm choice and model complexity, while industrial assets were more stable and forgiving, often yielding modest but consistent gains. These findings suggest that practitioners should consider not only the modeling technique but also the nature of the asset when designing forecasting systems. In summary, this project underscores the robustness and simplicity of LASSO as a forecasting tool in financial econometrics, particularly when data is noisy, limited, or structurally complex.

Reference

Gu, S., Kelly, B., & Xiu, D. (2020). Empirical asset pricing via machine learning.

Econometrica, 88(6), 2549–2577.

<https://academic.oup.com/rfs/article/33/5/2223/5758276>

Krauss, C., Do, X. A., & Huck, N. (2017). Deep neural networks, gradient-boosted

trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of*

Operational Research, 259(2), 689–702.

<https://www.sciencedirect.com/science/article/abs/pii/S0377221716308657>

Feng, G., Giglio, S., & Xiu, D. (2020). Taming the factor zoo: A test of new factors.

The Journal of Finance, 75(3), 1327–1370.

<https://onlinelibrary.wiley.com/doi/10.1111/jofi.12883>

Rasekhschaffe, K., & Jones, R. (2019). Machine learning for stock selection.

Financial Analysts Journal, 75(3). <https://ssrn.com/abstract=3330946>

Wirjanto, T. S. (2025). *AFM 423: Topics in financial econometrics*. University of Waterloo.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

<https://doi.org/10.1023/A:1010933404324>

Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–

22. <https://www.jstatsoft.org/article/view/v033i01>

Appendix 1: Scenario 1 - Market Portfolio Perspective

```
# Load required libraries
set.seed(423)
library(readr)
library(dplyr)
library(lubridate)
library(glmnet)
library(randomForest)
library(caret)
library(ggplot2)
library(fastshap)

# -----
# Download and Load Dataset
# -----
# The Fama-French Global 3-Factor CSV is automatically downloaded via download.file()
# and unzipped using read_csv() with a 6-line skip to remove metadata.
# Columns are renamed to include standard identifiers (Mkt-RF, SMB, HML),
# and the date column is parsed into proper Date format using the lubridate package.
# A response variable (target) is then defined as the lead (next-month) of the market excess return,
# aligning with the out-of-sample prediction goal.
temp <- tempfile()
base <- "http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/"
factor <- "Global_3_Factors"
format <- "_CSV.zip"
full_url <- paste(base, factor, format, sep = "")
download.file(full_url, temp, quiet = TRUE)
Global_3_Factors <- read_csv(unz(temp, "Global_3_Factors.csv"), skip = 6) %>%
  rename(date = `...1`) %>%
  mutate_at(vars(-date), ~ as.numeric(.)) %>%
  mutate(date = ymd(parse_date_time(date, "%Y%m", quiet = TRUE)))
# -----
# Prepare the Data
# -----
# Create a target variable: next month's Market excess return (Mkt-RF)
data <- Global_3_Factors %>%
  arrange(date) %>%
  mutate(target = lead(`Mkt-RF`)) %>% # Shift Mkt-RF by one period
  filter(!is.na(target), !is.na(date)) # Remove rows with NA in target or date
# -----
# Function for Train-Test Split
# -----
# A custom split_data() function is created to divide the data into training and test sets.
# Two splits are tested: 70/30 and 80/20, using a fixed random seed for reproducibility.
```

```

set.seed(123)
split_data <- function(df, ratio = 0.7) {
  n <- nrow(df)
  train_indices <- sample(seq_len(n), size = floor(ratio * n))
  list(train = df[train_indices, ],
       test = df[-train_indices, ])
}

# Three models are trained for performance comparison:
# (1) OLS (baseline model) via lm()
# (2) LASSO regression using glmnet::cv.glmnet(alpha = 1) with 10-fold CV to select lambda
# (3) Random Forest using randomForest() with ntree = 500, mtry = sqrt(p), maxnodes = 30
# All models are evaluated on the hold-out test set using:
# - Mean Squared Error (MSE)
# - Mean Absolute Error (MAE)
# - R-squared ( $R^2$ )
# These metrics are printed and summarized in a table for comparison.

# -----
# Research Question 1:
# Compare Baseline Linear Model vs. LASSO (70/30 Split)
# -----
split_70 <- split_data(data, ratio = 0.7)
train_70 <- split_70$train
test_70 <- split_70$test

# Define predictors and response for training/testing
x_train <- as.matrix(train_70 %>% select(SMB, HML))
y_train <- train_70$target
x_test <- as.matrix(test_70 %>% select(SMB, HML))
y_test <- test_70$target

# Baseline Linear Model (OLS)
lm_model <- lm(target ~ SMB + HML, data = train_70)
lm_pred <- predict(lm_model, newdata = test_70)
lm_mse <- mean((lm_pred - y_test)^2)
lm_mae <- mean(abs(lm_pred - y_test))
lm_r2 <- 1 - sum((lm_pred - y_test)^2) / sum((y_test - mean(y_test))^2)
lm_corr <- cor(lm_pred, y_test)
cat("Baseline LM MSE (70/30 split):", lm_mse, "\n")

## Baseline LM MSE (70/30 split): 20.67778

```

```

# LASSO Regression (using glmnet)
cv_lasso <- cv.glmnet(x_train, y_train, alpha = 1)
lasso_pred <- predict(cv_lasso, s = cv_lasso$lambda.min, newx = x_test)
lasso_mse <- mean((lasso_pred - y_test)^2)
lasso_mae <- mean(abs(lasso_pred - y_test))
lasso_r2 <- 1 - sum((lasso_pred - y_test)^2) / sum((y_test - mean(y_test))^2)
lasso_corr <- cor(lasso_pred, y_test)
cat("LASSO MSE (70/30 split):", lasso_mse, "\n")

## LASSO MSE (70/30 split): 20.35922

```



```

# -----
# Research Question 2:
# Compare LASSO vs. Random Forest (70/30 Split)
# -----
# Random Forest Model
rf_model <- randomForest(target ~ SMB + HML, data = train_70)
rf_pred  <- predict(rf_model, newdata = test_70)
rf_mse   <- mean((rf_pred - y_test)^2)
rf_mae   <- mean(abs(rf_pred - y_test))
rf_r2    <- 1 - sum((rf_pred - y_test)^2) / sum((y_test - mean(y_test))^2)
rf_corr  <- cor(rf_pred, y_test)
cat("Random Forest MSE (70/30 split):", rf_mse, "\n")

```

```
## Random Forest MSE (70/30 split): 22.5384
```

```

# -----
# Research Question 3:
# Impact of Train-Test Split Ratios Using LASSO (80/20 Split)
# -----
split_80 <- split_data(data, ratio = 0.8)
train_80 <- split_80$train
test_80  <- split_80$test

x_train80 <- as.matrix(train_80 %>% select(SMB, HML))
y_train80 <- train_80$target
x_test80  <- as.matrix(test_80 %>% select(SMB, HML))
y_test80  <- test_80$target

cv_lasso_80 <- cv.glmnet(x_train80, y_train80, alpha = 1)
lasso_pred80 <- predict(cv_lasso_80, s = cv_lasso_80$lambda.min, newx = x_test80)
lasso_mse80  <- mean((lasso_pred80 - y_test80)^2)
lasso80_mae  <- mean(abs(lasso_pred80 - y_test80))
lasso80_r2   <- 1 - sum((lasso_pred80 - y_test80)^2) / sum((y_test80 - mean(y_test80))^2)
lasso80_corr <- cor(lasso_pred80, y_test80)
cat("LASSO MSE (80/20 split):", lasso_mse80, "\n")

```

```
## LASSO MSE (80/20 split): 10.17995
```

```

# -----
# Summarize Results in a Table
# -----
results <- data.frame(
  Model = c("Baseline LM", "LASSO (70/30)", "Random Forest (70/30)", "LASSO (80/20)"),
  Train_Test_Ratio = c("70/30", "70/30", "70/30", "80/20"),
  MSE = c(lm_mse, lasso_mse, rf_mse, lasso_mse80),
  MAE = c(lm_mae, lasso_mae, rf_mae, lasso80_mae),
  R2 = c(lm_r2, lasso_r2, rf_r2, lasso80_r2),
  Corr = c(lm_corr, lasso_corr, rf_corr, lasso80_corr)
)
print(results)

```

```
##           Model Train_Test_Ratio      MSE      MAE      R2
```

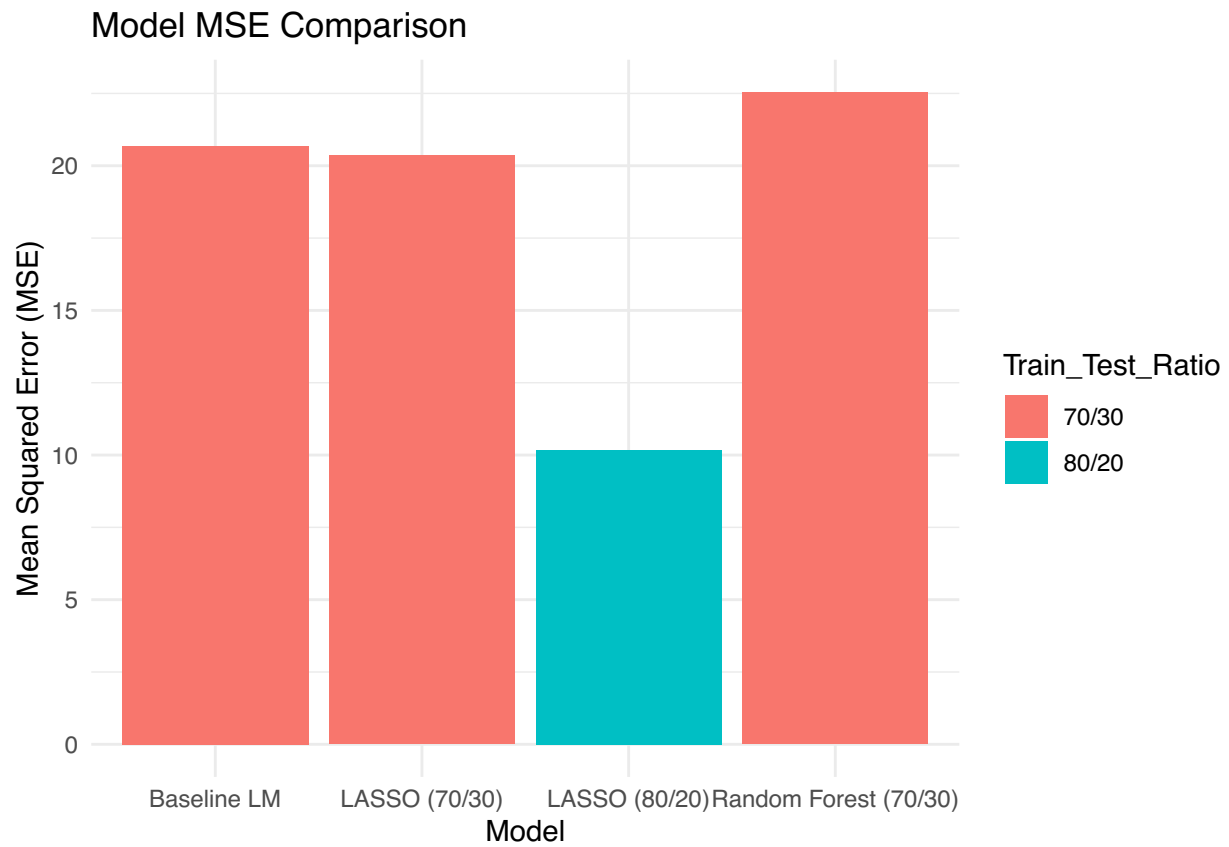
```
## 1      Baseline LM      70/30 20.67778 3.440686 -0.03842112
## 2      LASSO (70/30)    70/30 20.35922 3.408356 -0.02242359
## 3 Random Forest (70/30) 70/30 22.53840 3.560331 -0.13186020
## 4      LASSO (80/20)    80/20 10.17995 2.401493 -0.00498613
##      Corr
## 1 -0.08500365
## 2 -0.08189070
## 3 -0.07467516
## 4      NA
```

```
# -----
# Visualize Results: Bar Plots for MSE, R-squared, and MAE
# -----
# Results are visualized using ggplot2 as bar plots for MSE, MAE, and R2.
# This allows easy comparison across model types and train-test splits.
p1 <- ggplot(results, aes(x = Model, y = MSE, fill = Train_Test_Ratio)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  labs(title = "Model MSE Comparison",
       x = "Model", y = "Mean Squared Error (MSE)") +
  theme_minimal()

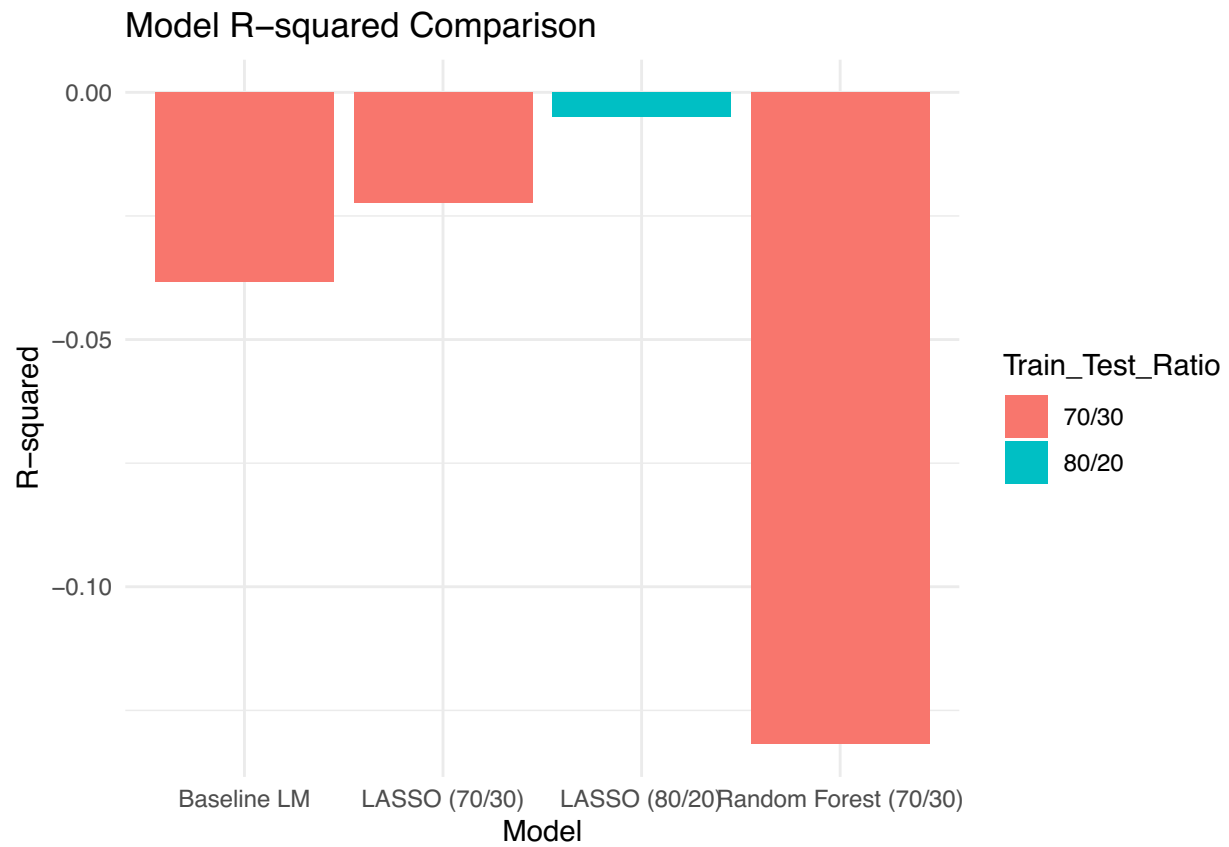
p2 <- ggplot(results, aes(x = Model, y = R2, fill = Train_Test_Ratio)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  labs(title = "Model R-squared Comparison",
       x = "Model", y = "R-squared") +
  theme_minimal()

p3 <- ggplot(results, aes(x = Model, y = MAE, fill = Train_Test_Ratio)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  labs(title = "Model MAE Comparison",
       x = "Model", y = "Mean Absolute Error (MAE)") +
  theme_minimal()

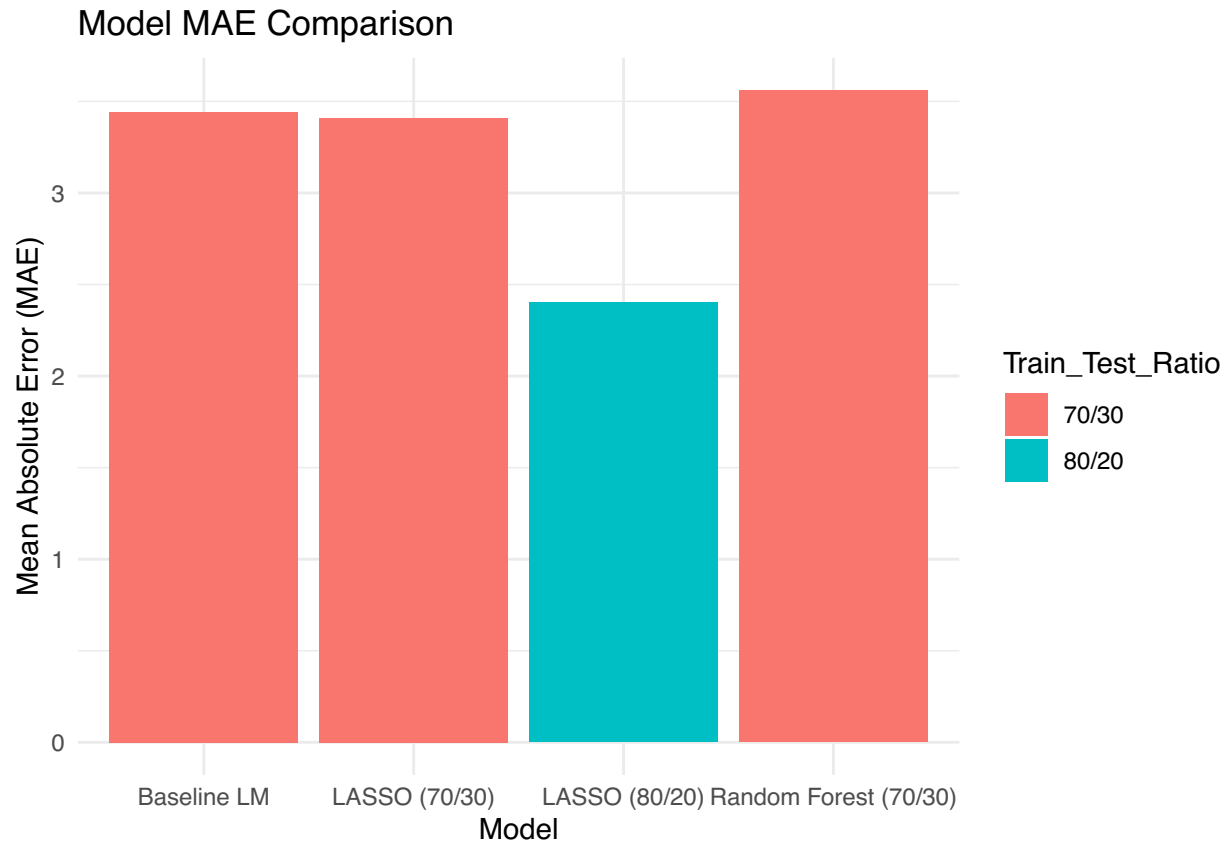
print(p1)
```



```
print(p2)
```



```
print(p3)
```

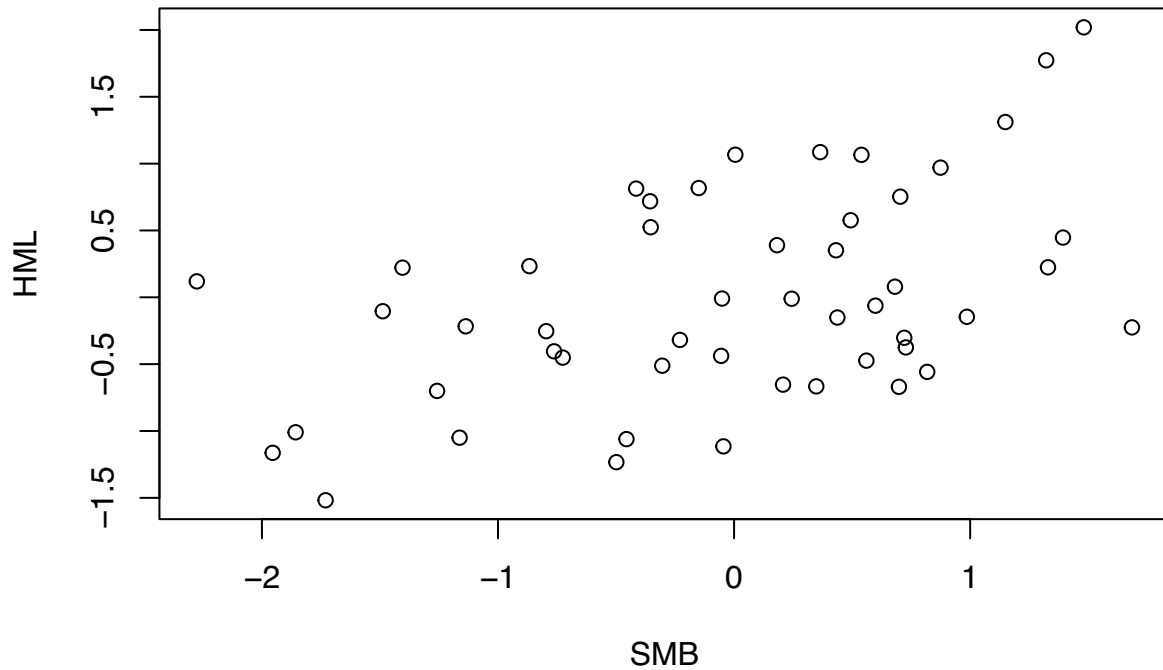


```
# -----
# SHAP Value Analysis for Model Interpretability
# -----
# To interpret model behavior, SHAP values are computed using the fastshap package:
# - For Random Forest: a prediction wrapper is used on the test subset
# - For LASSO: prediction extracted from glmnet.fit using best lambda
# SHAP scatter plots are shown to visualize feature influence.
# Define prediction wrappers for fastshap
# For Random Forest:
pred_rf <- function(model, newdata) {
  predict(model, newdata = newdata)
}
# For LASSO: Use the cv.glmnet object; extract predictions from the underlying glmnet.fit.
pred_lasso <- function(model, newdata) {
  as.vector(predict(model$glmnet.fit, newx = as.matrix(newdata), s = model$lambda.min))
}

# Compute SHAP values for a subset (first 50 rows) of test data for Random Forest
rf_sample <- test_70 %>% select(SMB, HML) %>% slice(1:50)
shap_rf <- explain(rf_model,
  X = rf_sample,
  pred_wrapper = pred_rf,
  nsim = 50) # Number of Monte Carlo simulations

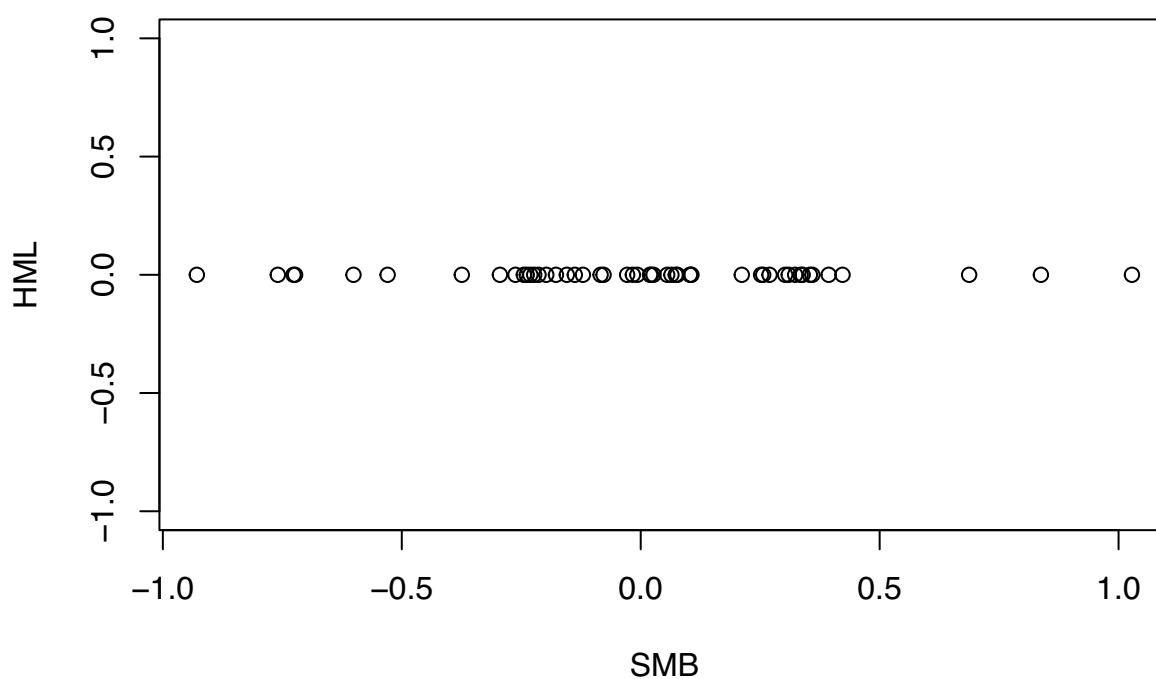
# Plot SHAP summary for Random Forest
plot(shap_rf, main = "SHAP Values - Random Forest")
```

SHAP Values – Random Forest



```
# Compute SHAP values for LASSO on the same sample of test data
lasso_sample <- test_70 %>% select(SMB, HML) %>% slice(1:50)
shap_lasso <- explain(cv_lasso,
                      X = as.matrix(lasso_sample),
                      pred_wrapper = pred_lasso,
                      nsim = 50)
# Plot SHAP summary for LASSO
plot(shap_lasso, main = "SHAP Values - LASSO")
```

SHAP Values – LASSO



```
# A scatter plot is created for LASSO predictions vs actual target values (70/30 split).  
# Includes a diagonal red dashed line to visualize perfect prediction.
```

```
# Scatter Plot for LASSO (70/30 Split): Predicted vs. Actual  
lasso_actual_vs_pred <- data.frame(Actual = y_test, Predicted = as.vector(lasso_pred))  
ggplot(lasso_actual_vs_pred, aes(x = Actual, y = Predicted)) +  
  geom_point(color = "blue", alpha = 0.6) +  
  geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "red") +  
  labs(title = "LASSO (70/30): Predicted vs. Actual",  
       x = "Actual Market Excess Return",  
       y = "Predicted Market Excess Return") +  
  theme_minimal()
```

LASSO (70/30): Predicted vs. Actual



Appendix 2: Scenario 2 - Index Perspective (Nasdaq vs Dow Jones)

```
library(readxl)
library(dplyr)
library(lubridate)
library(glmnet)
library(randomForest)
library(zoo)
library(Metrics)
library(ggplot2)
library(tidyr)

set.seed(423)
# Feature-engineering function (lags + 5-day rolling): Reads an index file,
# computes today's log-return and tomorrow's target and Adds lag-1, lag-2,
# and 5-day rolling-mean versions of every ratio and drops any rows
# with missing values

make_features <- function(path) {
  df <- read_excel(path) %>%
    rename(date = Dates, index = `Index Value`) %>%
    arrange(date) %>%
    # compute daily log-return and set next day as target
    mutate(
      log_ret = log(index/lag(index)),
      target = lead(log_ret)
    ) %>%
    # for each ratio column, add lag1, lag2, and 5-day rolling mean
    mutate(across(
      -c(date, index, log_ret, target),
      list(lag1 = ~lag(.,1),
           lag2 = ~lag(.,2),
           roll5 = ~rollapplyr(., width=5, mean, fill=NA)),
      .names = "{.col}_{.fn}"
    )) %>%
    # remove rows where log_ret, target, or any feature is NA
    filter(!is.na(log_ret), !is.na(target)) %>%
    drop_na()
  # extract feature matrix X, response y, and dates
  list(
    X = df %>% select(-date, -index, -log_ret, -target) %>% as.matrix(),
    y = df$target,
    dates = df$date
  )
}

# generate features for Dow Jones and Nasdaq
dow <- make_features("DOWJONES 10-Year Daily Return.xlsx")
ndq <- make_features("NASDAQ 10-Year Daily Return.xlsx")
```

```

# generate features for Dow Jones and Nasdaq
dow <- make_features("DOWJONES 10-Year Daily Return.xlsx")
ndq <- make_features("NASDAQ 10-Year Daily Return.xlsx")

# We loop over two splits (70/30 and 80/20),
# we split by time and fit OLS (baseline), LASSO (with CV), and Random Forest
# Then compute out-of-sample MSE, MAE, R2, and correlation

train_fracs <- c(0.7, 0.8)
all_stats <- list()

for(fr in train_fracs) {
  # split function (first fr% train, rest test)
  split_fun <- function(obj) {
    n <- nrow(obj$X)
    i <- floor(n * fr)
    list(
      Xtr = obj$X[1:i, ], ytr = obj$y[1:i],
      Xte = obj$X[(i+1):n, ], yte = obj$y[(i+1):n]
    )
  }
  dow_sp <- split_fun(dow)
  ndq_sp <- split_fun(ndq)

  # Function to fit and collect metrics
  fit_eval <- function(Xtr, ytr, Xte, yte) {
    # OLS
    df_tr <- as.data.frame(Xtr); df_tr$y <- ytr
    lm_mod <- lm(y~., data=df_tr)
    p_lm <- predict(lm_mod, newdata=as.data.frame(Xte))
    # LASSO
    cv_l <- cv.glmnet(Xtr, ytr, alpha=1)
    p_l <- predict(cv_l, s=cv_l$lambda.min, newx=Xte)
    # RF
    rf_mod <- randomForest(x=Xtr, y=ytr,
                          mtry=floor(sqrt(ncol(Xtr))),
                          ntree=500, maxnodes=30)
    p_rf <- predict(rf_mod, newdata=Xte)

    # collect metrics
    stats <- tibble::tibble(
      Model = c("OLS", "LASSO", "RandomForest"),
      MSE = c(mse(yte, p_lm), mse(yte, p_l), mse(yte, p_rf)),
      MAE = c(mae(yte, p_lm), mae(yte, p_l), mae(yte, p_rf)),
      R2 = c((1-sum((p_lm-yte)^2)/sum((yte-mean(yte))^2),
              1-sum((p_l -yte)^2)/sum((yte-mean(yte))^2),
              1-sum((p_rf-yte)^2)/sum((yte-mean(yte))^2)),
      Corr = c(cor(p_lm,yte), cor(p_l,yte), cor(p_rf,yte))
    )
    return(stats)
  }

  # Collect for Dow & Nasdaq

```

```

dow_stats <- fit_eval(dow_sp$Xtr, dow_sp$ytr, dow_sp$Xte, dow_sp$yte) %>%
  mutate(Index="Dow Jones", Split=fr)
ndq_stats <- fit_eval(ndq_sp$Xtr, ndq_sp$ytr, ndq_sp$Xte, ndq_sp$yte) %>%
  mutate(Index="Nasdaq", Split=fr)

all_stats[[as.character(fr)]] <- bind_rows(dow_stats, ndq_stats)
}

results <- bind_rows(all_stats)

# View numeric table
print(results)

```

```

## # A tibble: 12 x 7
##   Model      MSE      MAE      R2      Corr Index      Split
##   <chr>      <dbl>    <dbl>    <dbl>    <dbl> <chr>    <dbl>
## 1 OLS        0.000114  0.00786 -0.163     0.0407 Dow Jones 0.7
## 2 LASSO      0.0000985  0.00695 -0.000433 NA      Dow Jones 0.7
## 3 RandomForest 0.000122  0.00812 -0.244    -0.0439 Dow Jones 0.7
## 4 OLS        0.000242  0.0114  -0.0365     0.0287 Nasdaq    0.7
## 5 LASSO      0.000234  0.0110  -0.000348 NA      Nasdaq    0.7
## 6 RandomForest 0.000238  0.0112  -0.0188    -0.00145 Nasdaq    0.7
## 7 OLS        0.0000936  0.00701 -0.261     0.0681 Dow Jones 0.8
## 8 LASSO      0.0000742  0.00585 -0.0000256 NA      Dow Jones 0.8
## 9 RandomForest 0.0000819  0.00632 -0.104     0.0716 Dow Jones 0.8
## 10 OLS       0.000188  0.00983 -0.0740     0.0523 Nasdaq    0.8
## 11 LASSO     0.000175  0.00923  0.00108     0.0879 Nasdaq    0.8
## 12 RandomForest 0.000177  0.00934 -0.0107     0.0396 Nasdaq    0.8

```

```

# MSE by Model / Split / Index
ggplot(results, aes(x=Model, y=MSE, fill=factor(Split))) +
  geom_col(position="dodge") +
  facet_wrap(~Index) +
  labs(title="Out-of-Sample MSE by Model & Train/Split",
       fill="Train/Frac", y="MSE") +
  theme_minimal()

```



```
library(dplyr)
library(tidyr)

# Keep only the columns related for ΔMSE
imp <- results %>%
  select(Index, Split, Model, MSE) %>%

  # Pivot so that each row is one Index/Split
  pivot_wider(
    id_cols = c(Index, Split),
    names_from = Model,
    values_from = MSE
  ) %>%

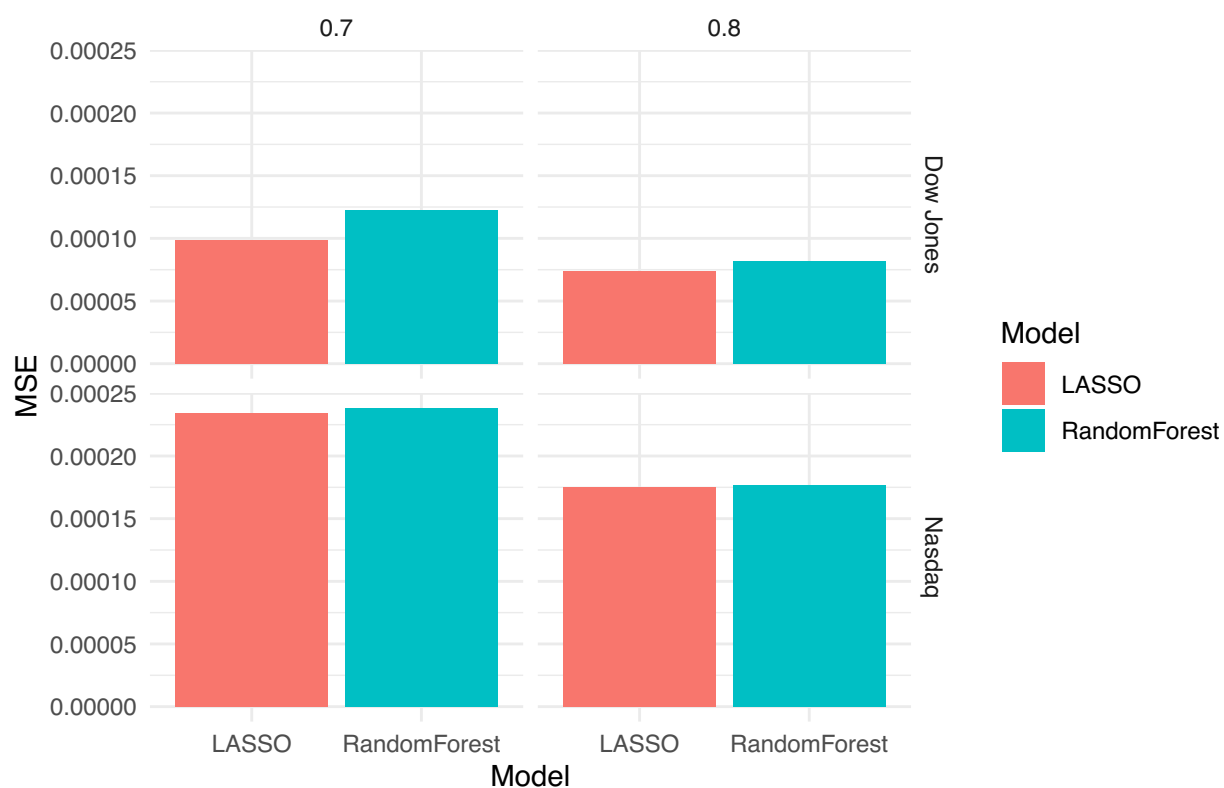
  # Compute Δ = MSE(OLS) - MSE(LASSO)
  mutate(Delta = OLS - LASSO)

ggplot(imp, aes(x=factor(Split), y=Delta, color=Index, group=Index)) +
  geom_line() + geom_point(size=3) +
  labs(title="LASSO Improvement over OLS (ΔMSE)",
       x="Train/Frac", y="MSE(OLS)-MSE(LASSO)") +
  theme_minimal()
```



```
# Compare LASSO vs RF directly
ggplot(results %>% filter(Model!="OLS"),
  aes(x=Model, y=MSE, fill=Model)) +
  geom_col() +
  facet_grid(Index~Split) +
  labs(title="MSE: LASSO vs Random Forest",
    y="MSE") +
  theme_minimal()
```

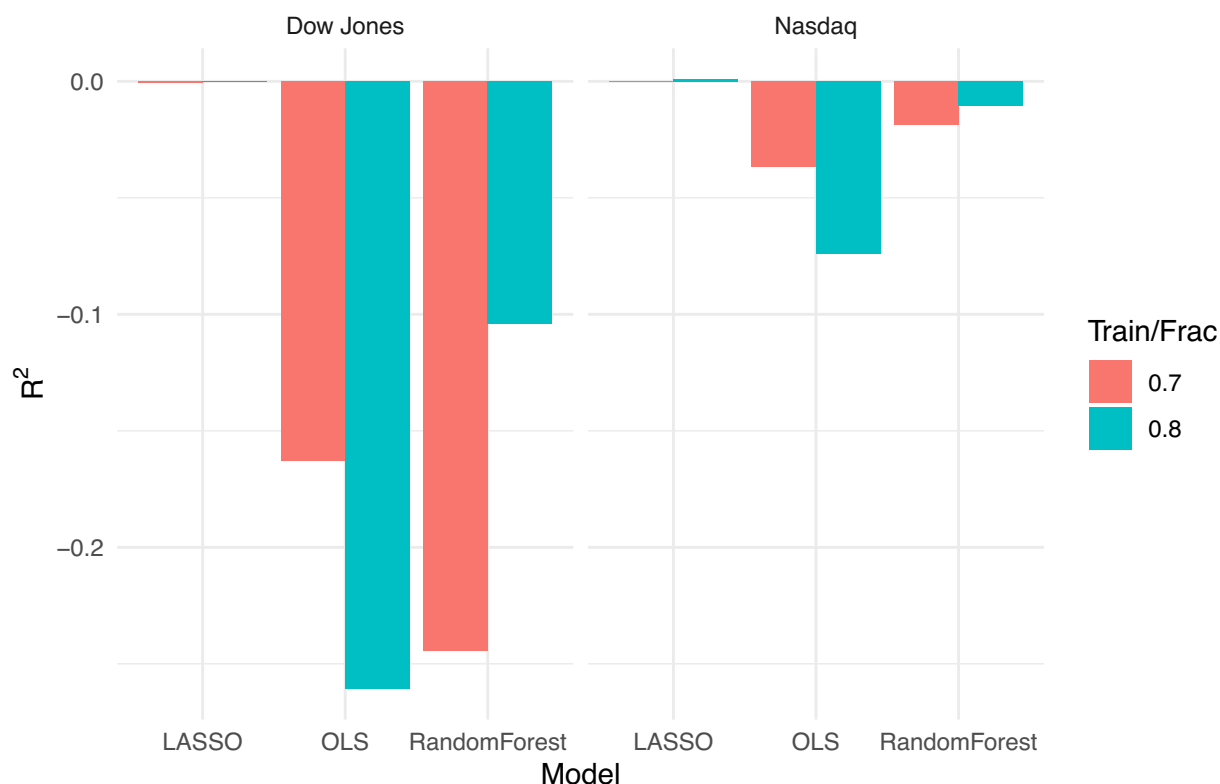
MSE: LASSO vs Random Forest



```
library(dplyr)
library(tidyr)
library(ggplot2)

# Bar chart of R-squared by Model / Split / Index
ggplot(results, aes(x=Model, y=R2, fill=factor(Split))) +
  geom_col(position="dodge") +
  facet_wrap(~Index) +
  labs(
    title = "Out-of-Sample R2 by Model & Train/Split",
    x = "Model",
    y = expression(R2),
    fill = "Train/Frac"
  ) +
  theme_minimal()
```

Out-of-Sample R^2 by Model & Train/Split



```
# Compute  $\Delta R^2 = R^2(\text{LASSO}) - R^2(\text{OLS})$ 
imp_R2 <- results %>%
  select(Index, Split, Model, R2) %>%
  pivot_wider(
    id_cols = c(Index, Split),
    names_from = Model,
    values_from = R2
  ) %>%
  mutate(Delta_R2 = LASSO - OLS)

print(imp_R2)
```

```
## # A tibble: 4 x 6
##   Index      Split      OLS      LASSO RandomForest Delta_R2
##   <chr>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
## 1 Dow Jones  0.7 -0.163 -0.000433 -0.244  0.162
## 2 Nasdaq    0.7 -0.0365 -0.000348 -0.0188  0.0361
## 3 Dow Jones  0.8 -0.261 -0.000256 -0.104  0.261
## 4 Nasdaq    0.8 -0.0740  0.00108 -0.0107  0.0750
```

```
# 3) Plot  $\Delta R^2$ 
ggplot(imp_R2, aes(x=factor(Split), y=Delta_R2, color=Index, group=Index)) +
  geom_point(size=3) +
  geom_line() +
  labs(
    title = expression("LASSO Improvement over OLS ("*Delta*R^2*")"),
    x = "Train Fraction",
    y = expression(R^2[LASSO] - R^2[OLS])) +
```

```
theme_minimal()
```



Appendix 3: Scenario 3 - Individual Stock Perspective (TSLA vs KO)

```
library(readxl)
library(dplyr)
library(lubridate)
library(glmnet)
library(randomForest)
library(zoo)
library(caret)
library(Metrics)
library(ggplot2)
library(tidyr)

set.seed(423)
make_features <- function(path) {
  df <- readxl::read_excel(path) %>%
    # 1) Rename your actual columns to `date` and `index`
    dplyr::rename(
      date = Dates,
      index = `Share Pricing`
    ) %>%
    dplyr::arrange(date) %>% # ensure chronological order
    # 2) Compute today's log-return and tomorrow's return as the target
    dplyr::mutate(
      log_ret = log(index / dplyr::lag(index)),
      target = dplyr::lead(log_ret)
    ) %>%
    # 3) For every other column (the ratios), create lag1, lag2, and 5-day rolling mean
    dplyr::mutate(across(
      ~c(date, index, log_ret, target),
      list(
        lag1 = ~dplyr::lag(., 1),
        lag2 = ~dplyr::lag(., 2),
        roll5 = ~zoo::rollapply(., width = 5, mean, fill = NA)
      ),
      .names = "{.col}_{.fn}"
    )) %>%
    # 4) Drop any rows missing returns, target, or engineered features
    dplyr::filter(!is.na(log_ret), !is.na(target)) %>%
    tidyr::drop_na()

  # 5) Return:
  #   - X: matrix of all ratio-based features
  #   - y: next-day log-return
  #   - dates: the date vector for plotting
  list(
    X = df %>% dplyr::select(-date, -index, -log_ret, -target) %>% as.matrix(),
    y = df$target,
    dates = df$date
  )
}
```

```

    dates = df$date
  )
}

# Apply the function to both TSLA and KO:
tsla <- make_features("TSLA 10-Year Daily Return.xls")
ko    <- make_features("KO 10-Year Daily Return.xls")

# Train/Test Split & Model Fitting
train_fractions <- c(0.7, 0.8)
all_stats <- list()

# helper: time-based split
split_fun <- function(obj, frac) {
  n <- nrow(obj$X)
  i <- floor(n * frac)
  list(
    Xtr = obj$X[1:i, ], ytr = obj$y[1:i],
    Xte = obj$X[(i+1):n, ], yte = obj$y[(i+1):n]
  )
}

# helper: fit OLS, LASSO, RF; compute MSE, MAE, R2, Corr
fit_eval <- function(Xtr, ytr, Xte, yte) {
  # OLS baseline
  df_tr <- as.data.frame(Xtr); df_tr$y <- ytr
  lm_mod <- lm(y ~ ., data = df_tr)
  p_lm <- predict(lm_mod, newdata = as.data.frame(Xte))

  # LASSO with CV
  cv_l <- cv.glmnet(Xtr, ytr, alpha = 1)
  p_l <- predict(cv_l, s = cv_l$lambda.min, newx = Xte)

  # Random Forest
  rf_mod <- randomForest(
    x = Xtr,
    y = ytr,
    mtry = floor(sqrt(ncol(Xtr))),
    ntree = 500,
    maxnodes = 30,
    importance = TRUE
  )
  p_rf <- predict(rf_mod, newdata = Xte)

  # collect metrics
  tibble::tibble(
    Model = c("OLS", "LASSO", "RandomForest"),
    MSE = c(mse(yte, p_lm), mse(yte, p_l), mse(yte, p_rf)),
    MAE = c(mae(yte, p_lm), mae(yte, p_l), mae(yte, p_rf)),
    R2 = c(
      1 - sum((p_lm - yte)^2)/sum((yte - mean(yte))^2),
      1 - sum((p_l - yte)^2)/sum((yte - mean(yte))^2),
      1 - sum((p_rf - yte)^2)/sum((yte - mean(yte))^2)
    )
  )
}

```

```

    ),
    Corr = c(cor(p_lm, yte), cor(p_l, yte), cor(p_rf, yte))
  )
}

# loop over train_fracs and symbols
symbols <- list("TSLA" = tsla, "KO" = ko)

for (fr in train_fracs) {
  for (name in names(symbols)) {
    obj <- symbols[[name]]
    sp <- split_fun(obj, fr)
    stats <- fit_eval(sp$Xtr, sp$ytr, sp$Xte, sp$yte) %>%
      mutate(Symbol = name, Split = fr)
    all_stats[[paste(name, fr, sep = "_")] ] <- stats
  }
}

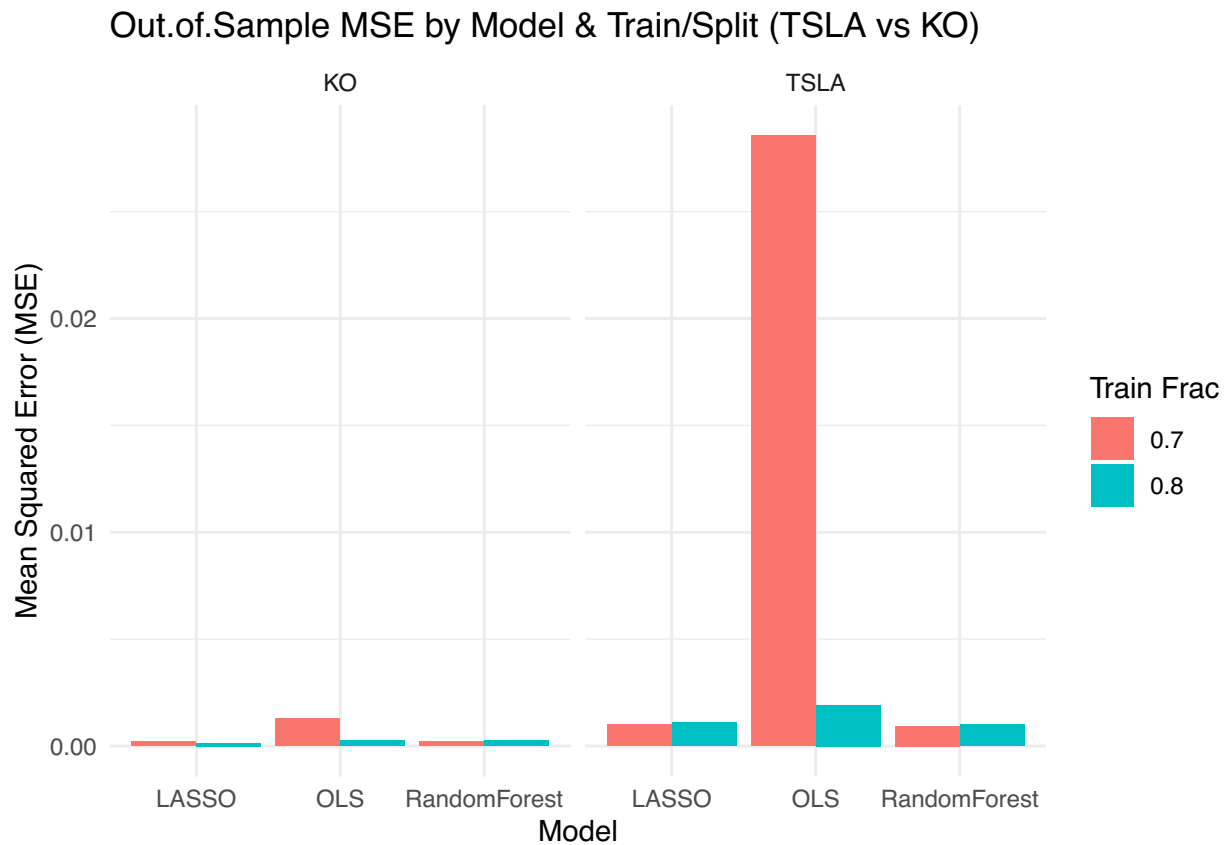
# combine results
tsla_ko_results <- bind_rows(all_stats)
print(tsla_ko_results)

## # A tibble: 12 x 7
##   Model      MSE      MAE      R2      Corr Symbol Split
##   <chr>    <dbl>    <dbl>    <dbl>    <dbl> <chr>  <dbl>
## 1 OLS      0.0286    0.161   -33.0   -0.0569 TSLA    0.7
## 2 LASSO    0.000992  0.0243   -0.182  -0.109   TSLA    0.7
## 3 RandomForest 0.000940  0.0239   -0.120  -0.0833 TSLA    0.7
## 4 OLS      0.00128    0.0268  -10.4    0.0549  KO      0.7
## 5 LASSO    0.000205  0.0110   -0.818   0.0741  KO      0.7
## 6 RandomForest 0.000219  0.0117   -0.941  -0.000722 KO      0.7
## 7 OLS      0.00192    0.0345   -1.34   -0.0730 TSLA    0.8
## 8 LASSO    0.00109    0.0250   -0.330  -0.0669 TSLA    0.8
## 9 RandomForest 0.00103    0.0244   -0.254  -0.0903 TSLA    0.8
## 10 OLS     0.000273  0.0107   -1.18    0.0407  KO      0.8
## 11 LASSO   0.000142  0.00806  -0.131   0.0111  KO      0.8
## 12 RandomForest 0.000254  0.0132   -1.02    0.0808  KO      0.8

# Visualization

# MSE by Model & Split
ggplot(tsla_ko_results, aes(x = Model, y = MSE, fill = factor(Split))) +
  geom_col(position = "dodge") +
  facet_wrap(~Symbol) +
  labs(
    title = "Out-of-Sample MSE by Model & Train/Split (TSLA vs KO)",
    x = "Model",
    y = "Mean Squared Error (MSE)",
    fill = "Train Frac"
  ) +
  theme_minimal()

```

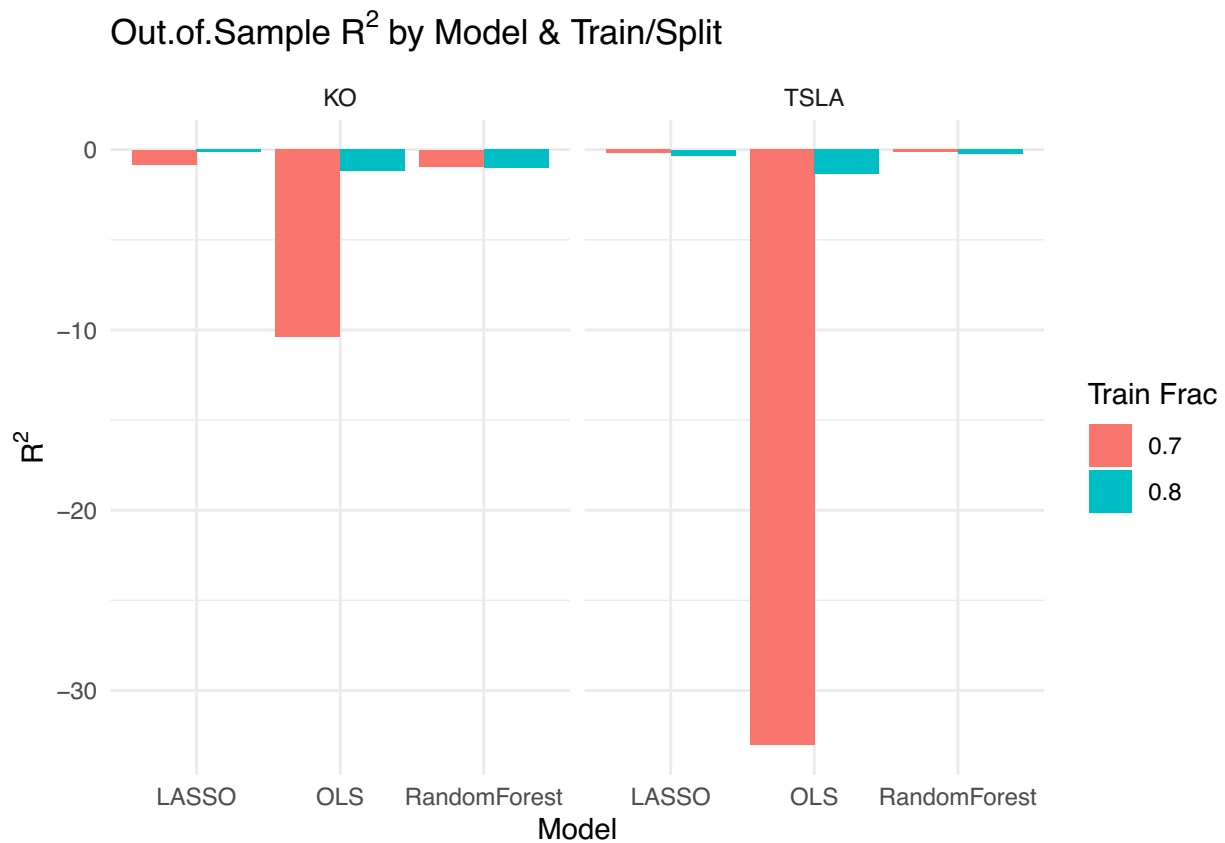


```
# LASSO Improvement over OLS ( $\Delta$ MSE)
imp <- tsla_ko_results %>%
  select(Symbol, Split, Model, MSE) %>%
  pivot_wider(
    id_cols = c(Symbol, Split),
    names_from = Model,
    values_from = MSE
  ) %>%
  mutate(Delta = OLS - LASSO)

ggplot(imp, aes(x = factor(Split), y = Delta, color = Symbol, group = Symbol)) +
  geom_line() +
  geom_point(size = 3) +
  labs(
    title = "LASSO Improvement over OLS ( $\Delta$ MSE)",
    x = "Train Fraction",
    y = expression(MSE[OLS] - MSE[LASSO])
  ) +
  theme_minimal()
```



```
# R2 Comparison & ΔR2
# (a) R2 by Model
ggplot(tsla_ko_results, aes(x = Model, y = R2, fill = factor(Split))) +
  geom_col(position = "dodge") +
  facet_wrap(~Symbol) +
  labs(
    title = expression("Out-of-Sample " * R^2 * " by Model & Train/Split"),
    x      = "Model",
    y      = expression(R^2),
    fill   = "Train Frac"
  ) +
  theme_minimal()
```



```
# (b)  $\Delta R^2$ 
imp_R2 <- tsla_ko_results %>%
  select(Symbol, Split, Model, R2) %>%
  pivot_wider(
    id_cols = c(Symbol, Split),
    names_from = Model,
    values_from = R2
  ) %>%
  mutate(Delta_R2 = LASSO - OLS)

ggplot(imp_R2, aes(x = factor(Split), y = Delta_R2, color = Symbol, group = Symbol)) +
  geom_point(size = 3) +
  geom_line() +
  labs(
    title = expression("LASSO Improvement over OLS (" * Delta * R^2 * ")"),
    x = "Train Fraction",
    y = expression(R^2[LASSO] - R^2[OLS])
  ) +
  theme_minimal()
```

